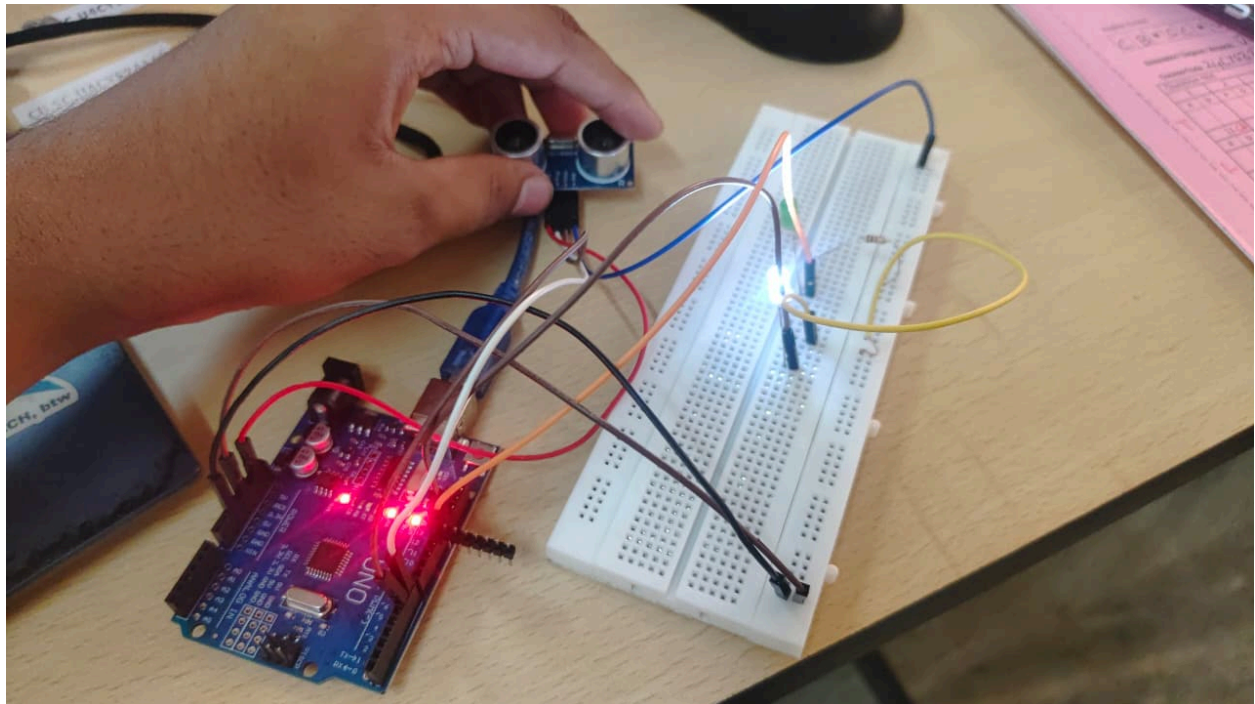


Praneesh R V
CB.SC.U4CYS23036
24CYS333 Endsemester



```
#include <Arduino.h>
const int HEIGHT = 100;
const int LOW_T = 50;
const int HIGH_T = 90;

long getDistance() {
  digitalWrite(7, LOW);
  delayMicroseconds(2);
  digitalWrite(7, HIGH);
  delayMicroseconds(10);
  digitalWrite(7, LOW);
  long time = pulseIn(6, HIGH);
  return time * 0.034 / 2;
```

```

}

void setup() {
  Serial.begin(9600);
  pinMode(7, OUTPUT);
  pinMode(6, INPUT);
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
}

void loop() {
  long distance = getDistance();
  int waterHeight = HEIGHT - distance;
  if (waterHeight < 0) waterHeight = 0;
  float percentage = ((float)waterHeight / HEIGHT) * 100;

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.print("Water Level: ");
  Serial.print(percentage);

  if (percentage <= LOW_T) {
    Serial.println("LOW WATER LEVEL!");
    digitalWrite(12, HIGH);
    digitalWrite(13, LOW);
  }
  else if (percentage >= HIGH_T) {
    Serial.println("TANK NEARLY FULL!");
    digitalWrite(12, LOW);
    digitalWrite(13, HIGH);
  }
  else {
    Serial.println("Level Normal");
    digitalWrite(12, LOW);
    digitalWrite(13, LOW);
  }
  delay(1000);
}

```

Gemini chat link for dashboard generation

<https://gemini.google.com/share/3d51493441e9>

cat dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Water Level Dashboard</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center;
background-color: #f4f4f9; padding: 50px; }
    .container { max-width: 400px; margin: auto; background: white;
padding: 20px; border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}
    .tank-container { width: 100px; height: 300px; border: 4px solid
#333; border-radius: 5px; margin: 20px auto; position: relative;
background: #e0e0e0; overflow: hidden; }
    .water { position: absolute; bottom: 0; width: 100%; background:
#3498db; transition: height 0.5s ease; }
    button { padding: 10px 20px; font-size: 16px; cursor: pointer;
background: #2ecc71; color: white; border: none; border-radius: 5px; }
    .data-text { font-size: 18px; margin: 10px 0; font-weight: bold; }
    .status-low { color: red; }
    .status-normal { color: green; }
    .status-full { color: orange; }
  </style>
</head>
<body>

<div class="container">
  <h2>Water Monitor</h2>
  <button id="connectBtn">Connect to Arduino</button>

  <div class="tank-container">
    <div class="water" id="waterFill" style="height: 0%;"></div>
  </div>

  <div class="data-text">Level: <span id="levelText">0</span>%</div>
  <div class="data-text">Distance: <span id="distText">0</span> cm</div>
  <div class="data-text">Status: <span id="statusText">-</span></div>
</div>
```

```

<script>
  let port;
  let reader;

  document.getElementById('connectBtn').addEventListener('click', async
() => {
    try {
      // Request the serial port
      port = await navigator.serial.requestPort();
      await port.open({ baudRate: 9600 });
      document.getElementById('connectBtn').innerText = "Connected";
      document.getElementById('connectBtn').style.background =
"#95a5a6";

      // Set up the reader
      const textDecoder = new TextDecoderStream();
      const readableStreamClosed =
port.readable.pipeTo(textDecoder.writable);
      reader = textDecoder.readable.getReader();

      let buffer = "";

      while (true) {
        const { value, done } = await reader.read();
        if (done) {
          reader.releaseLock();
          break;
        }

        buffer += value;
        let lines = buffer.split('\n');

        // Keep the last incomplete line in the buffer
        buffer = lines.pop();

        for (let line of lines) {
          processData(line.trim());
        }
      }
    } catch (err) {
      console.error("Error connecting to serial port:", err);
      alert("Connection failed or was cancelled. Make sure your
Arduino serial monitor is closed.");
    }
  });

  function processData(data) {

```

```

// Expecting format: Distance,Percentage,Status
let parts = data.split(',');
if (parts.length === 3) {
    let distance = parseFloat(parts[0]);
    let percentage = parseFloat(parts[1]);
    let status = parts[2].trim();

    // Cap percentage between 0 and 100 for the visual
    let visualPercent = Math.max(0, Math.min(100, percentage));

    // Update DOM
    document.getElementById('waterFill').style.height =
visualPercent + "%";
    document.getElementById('levelText').innerText =
percentage.toFixed(1);
    document.getElementById('distText').innerText = distance;

    let statusEl = document.getElementById('statusText');
    statusEl.innerText = status;

    // Update colors based on status
    statusEl.className = "";
    if (status === "LOW") statusEl.classList.add("status-low");
    else if (status === "NORMAL")
statusEl.classList.add("status-normal");
    else if (status === "FULL")
statusEl.classList.add("status-full");
    }
}
</script>

</body>
</html>

```